

컴퓨터공학과 4학년 박민수

코드가 운영 환경을 만났을 때

예상치 못한 동시성과 AI의 함정



목차

단순 기능 구현을 넘어
안전한 운영 아키텍처로

- 자기소개
- 로컬 환경의 착각과 운영 서버의 민낯
- 첫번째 충돌 : 동시성 문제와의 전쟁
- 두번째 충돌 : AI 코드의 환상과 위협
- 운영 서버에서 살아남기
- 진화하는 시대의 코어 역량

자기소개

컴퓨터공학과 4학년 박민수

제로베이스 백엔드 스쿨 : 실시간 주차장 정보 서비스 개발, 원데이 클래스 플랫폼 개발

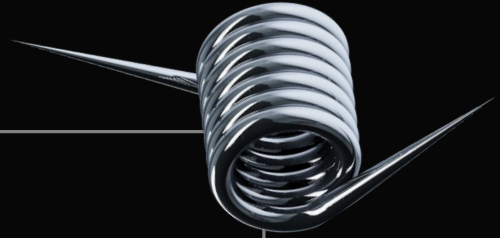
명지대 보안 동아리 MJSEC : 개발팀 백엔드 멘토, CTF 대회 사이트 개발, 동아리 LMS 개발

방산 안보 연구소 : MCP 취약점 분석, LIG Nex1 과제 참여 및 CMMC Compliance Tool 개발

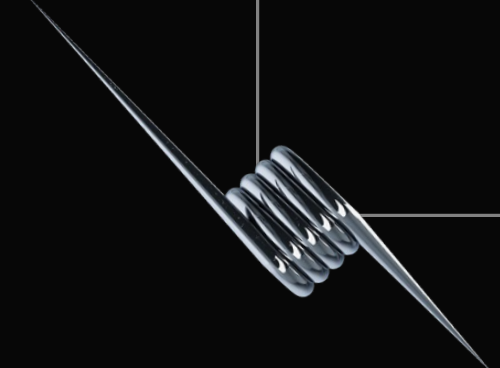


로컬 환경의 착각과 운영 환경의 민낯

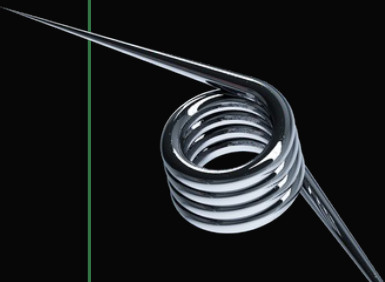




Local

- 평소 과제나 사이드 프로젝트를 진행하는 로컬 환경은 완벽하게 통제된 온실과 같음
 - 사용자는 개발자 본인 1명
 - 네트워크 지연은 0에 가까움
 - 비즈니스 로직이 의도대로 동작하는지만 확인하면 테스트 통과
- 

Prod

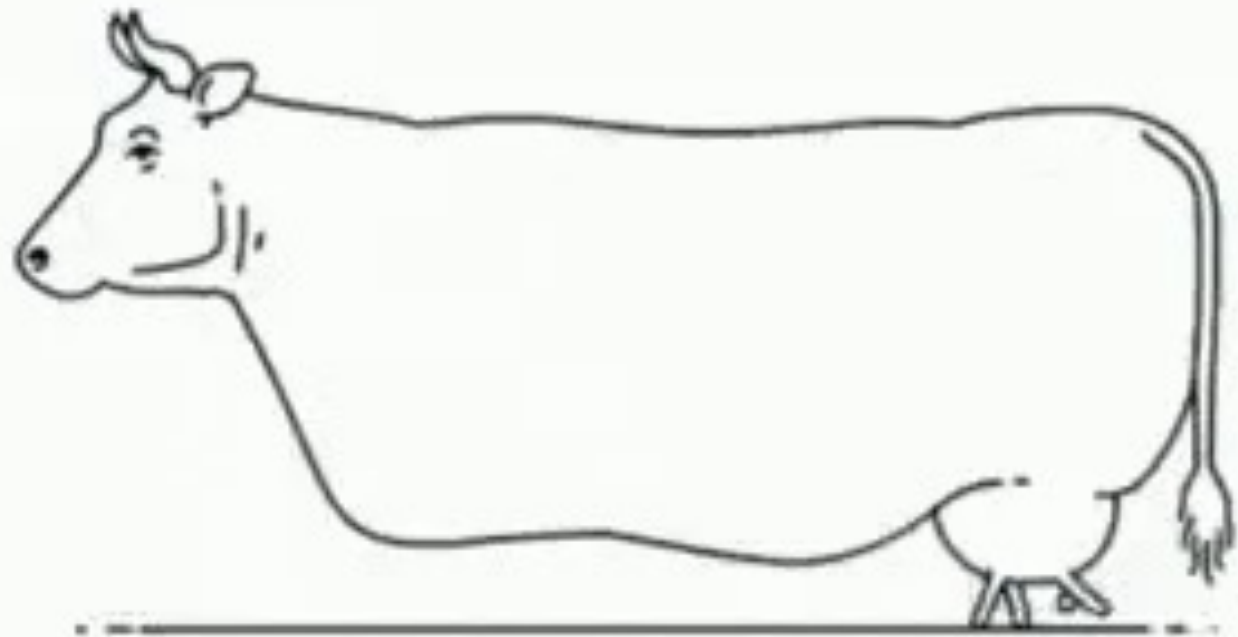


- '기능이 동작하는가'는 아주 기본적인 전제
- 유저들은 상상도 못한 방법으로 예외를 발생시킴
- 상태를 어떻게 관리하고, 폭증하는 트래픽을 어떻게 통제할 것인가
- 운영 비용을 어떻게 절감시킬 것인가

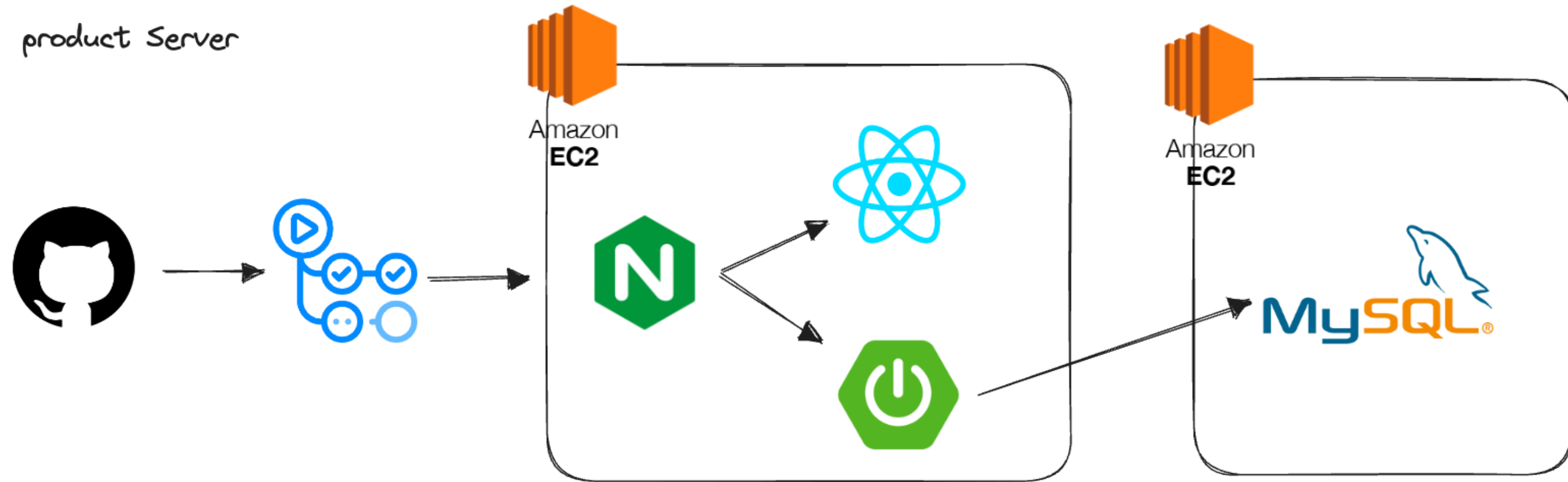


Fullstack HQ
@fullstackhq

1st rule of programming: If it works
don't touch it. 🐱

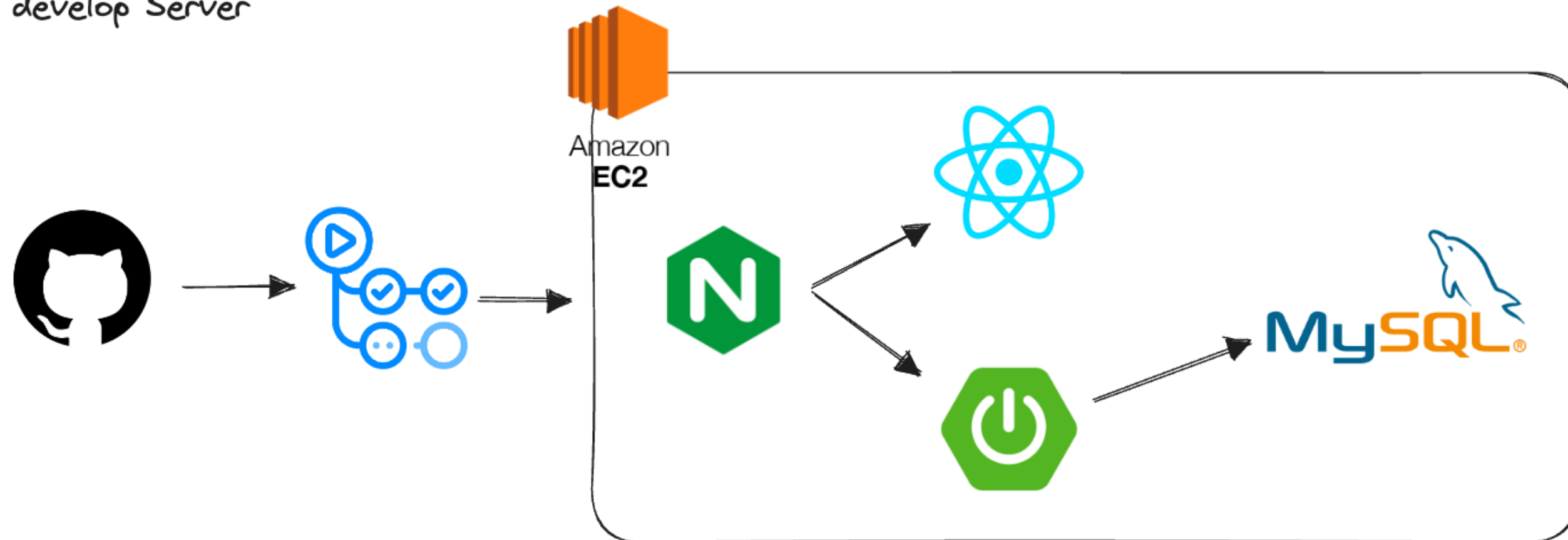


product Server



비용, 확장성, 트래픽 처리 등...

develop Server



첫번째 충돌 : 동시성 문제와의 전쟁



CTF

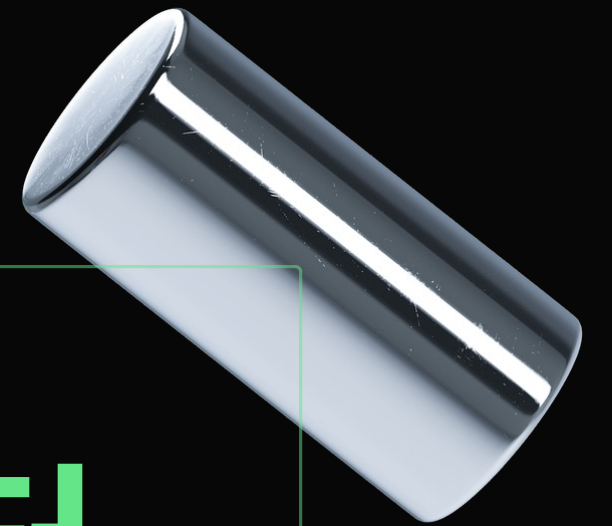
정보보안 분야의 해킹 방어 대회

주최 측이 마련한 시스템이나 소프트웨어에 숨겨진 취약점(웹 해킹, 리버스 엔지니어링, 암호학 등)을 찾아내어 Flag라고 불리는 특정 문자열(정답)을 탈취하고 이를 서버에 제출하여 점수를 획득

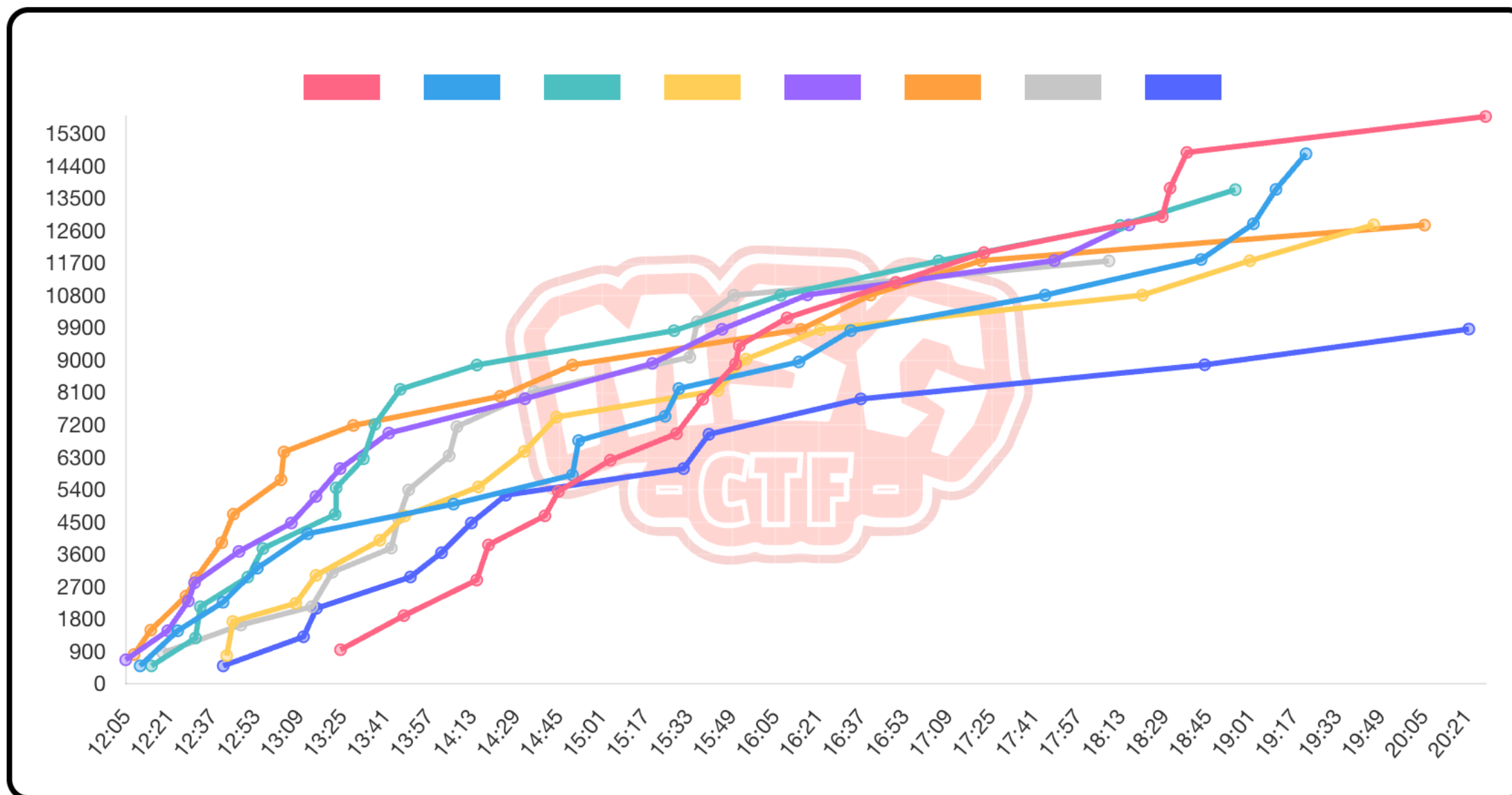
참가자들은 1점이라도 먼저 얻기 위해 자동화된 파이썬 스크립트 등을 작성하여 0.01초 단위로 수십, 수백 번의 정답 제출 요청을 서버에 폭격



동시성 문제와 악의적인 데이터
주입 공격이 실시간으로
발생하는 매우 특수한 운영환경



SCOREBOARD



인서 여친 구함...

플래그에 미원 들통...

야호행복포포암...

개웃겨서클레이남을...

Do you wanna...

똥냐?

갯잎

안양시 수지구...



Scoreboard

Challenge

Ranking



Forensics

Crypto

Pwn

Android

Reversing

Misc

Web

Signature

Clear

Select All

PWN

MSG Corp_SYSTEM??

957

M30

FORENSICS

Internet Activity Tracking

995

M30

REV

Alarm

676

M30

PWN

Format Sniper Revenge

1000

M150

PWN

Note

949

M30

WEB

SAKURA STUDIO

775

M30

Signature

securious

0

M90

Signature

SecurityFirst

0

M90

REV

Decryption

1000

M150

PWN

Wish List

977

M30



Scoreboard

Challenge

Ranking



OneCard

REV

🏆 11팀 해결

슈퍼 컴퓨팅 능력을 가진 이 원카드 AI는 게임에서 저본적이 없습니다.
속임수라도 사용하고 있는걸까요?

기본 정보

오답 3회 시 · 30초 클다운

↓ 첨부 다운로드

플래그 제출

FLAG 입력

제출

CTF CONCURRENCY ATTACK: RACE CONDITION DIAGRAM

HACKER'S SCRIPT



REQUEST 1 (FLAG SUBMIT)

REQUEST 2 (FLAG SUBMIT)

점수 중복 지급

1. 스크립트로 동시에 제출 요청
2. 스레드 간 경쟁
3. 100점 + 100점
4. 점수 중복 획득 발생

시스템 정합성 파괴

CTF BACKEND SERVER

(Application Layer)

THREAD A
(REQUEST 1)

THREAD B
(REQUEST 2)

SELECT SOLVED FROM user_flag
WHERE user_id=1, challenge_id=1



RACE WINDOW!

SELECT SOLVED FROM
user_flag WHERE user_id=1,
challenge_id=1

UPDATE user_flag
SET solved=True, points=+100
WHERE ...

DUPLICATE PAYOUT

UPDATE user_flag
SET solved=True, points=+100
WHERE ...

RESULT: DUPLICATE POINTS AWARDED!
(SCORE: 200 POINTS FOR 1 CHALLENGE)

DATABASE

(Data Persistence Layer)

Result: False (Not Solved)

Result: False (Not Solved)

Points added correctly
for the first time

Result: False
Points add* again

문제 상황

Read – Modify – Write

정답 플래그 제출 시 백엔드는 문제 풀이 여부 조회, 정답 판단, 점수 업데이트의 3단계를 거침

스크립트를 통한 동시 다발적 요청이 들어오면, 점수가 업데이트 되기 직전의 미세한 컨텍스트 스위칭이라는 틈이 발생함

여러 스레드가 동시에 ‘아직 풀지 않은 상태’를 읽어가게 되어, 한 유저가 단일 문제로 다중 점수를 획득하는 치명적인 레이스 컨디션 공격이 이루어짐

해결 과정

Redis Distributed Lock

DB 비관적 락 (트랜잭션 강제 직렬화)

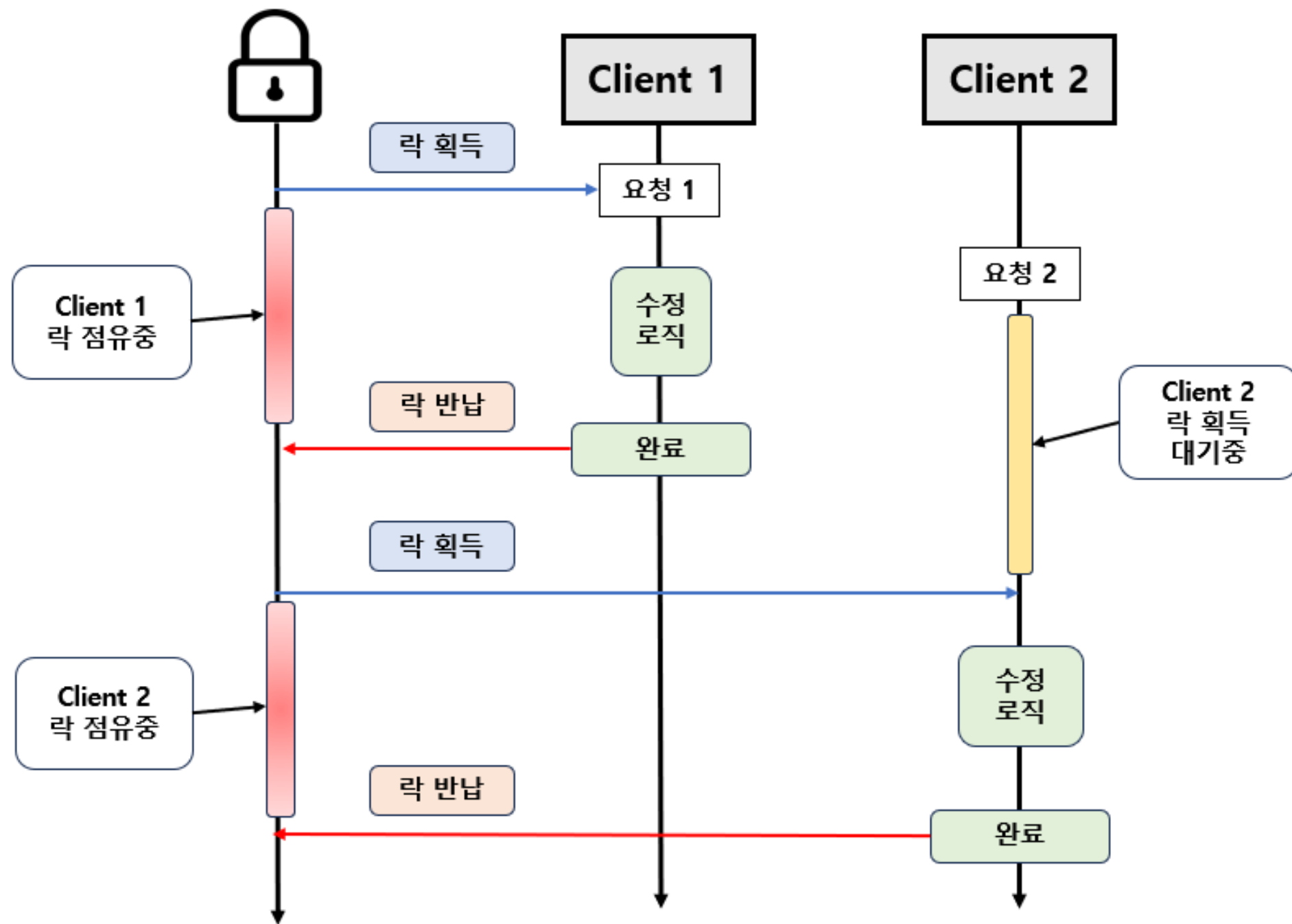
데이터 무결성은 확실하지만 트랜잭션 점유 시간이 길어져,
CTF처럼 충돌이 극심한 상황에서는 커넥션 풀 고갈 및 전체 시스템
데드락 위험이 극대화 됨

DB 낙관적 락 (버전 비교)

충돌 감지 시 개발자가 Retry 로직을 짜야 하는데, 스크립트 공격
상황에서는 무한 재시도가 발생해 서버의 CPU 부하가
기하급수적으로 치솟음

Redis 분산 락 도입

싱글 스레드와 인메모리 기반의 고속 I/O 특성을 활용하여
애플리케이션과 DB 사이에 방어막 구축



Spring Boot에서는 락을 구현하는 데에 필요한
Lettuce, Redisson 라이브러리 제공

Lettuce – Spin Lock

계속해서 요청을 보내는 방식으로 인해 Redis에
부하가 생길 수 있다는 단점이 있음

Redisson – Pub/Sub

락이 해제 되면 Subscribe 중인 클라이언트에게
알림을 보내기 때문에 부하가 발생하지 않음

두번째 충돌 : AI 코드의 환상과 위협



AI 생성 코드 편하긴 하지만...

새로운 위협의 대두



현재는 AI가 비즈니스 로직을
자동 생성하는 시대로 진입



빠른 기능 구현에 탁월



전체 시스템의 보안
컨텍스트와 아키텍처 원칙을
무시하는 경향이 있음

위협 사례 1

Spring JPA 환경의 권한 상승

AI가 보일러 플레이트 코드를 줄이기 위해 외부 입력을 받는 DTO를 생략하고 DB와 직결된 Entity를 컨트롤러에 직접 매핑하는 코드를 생성함

악의적 유저가 회원 정보 수정 JSON 페이로드에 “role”: “ADMIN”이라는 필드를 주입함

JPA가 필터링 없이 DB에 값을 업데이트하여 일반 유저가 단 한 줄의 코드로 관리자 권한을 탈취하는 대참사 발생

```
@RestController
public class UserController {

    @Autowired
    private UserRepository userRepository;

    @PutMapping("/users/{id}")
    public User updateProfile(@PathVariable Long id, @RequestBody User user) {
        user.setId(id);
        return userRepository.save(user);
    }
}
```

Vulnerability Exploitation and Data Flow Diagram

Hacker's Malicious JSON Payload

```
{
  "email": "hacker@evil.com",
  "firstName": "Attacker",
  "lastName": "User",
  "bio": "Compromised user profile",
  "role": "ADMIN" // Crucial injected value
}
```

"role": "ADMIN"

Vulnerability: Injected Admin Role

Privilege Escalation Data Flow Visualization

Attacker sends
malicious packet



Attacker / Malicious User

Vulnerable Server: UserController

Request handling (UserController)

Memory Area

User Entity

user_id	name
1	hacker
role	"ADMIN"
...	

RequestBody User user

Vulnerable Direct Mapping
(dto omitted) -> Entity (User user)

SELECT

JPA Repository
(JPA Repository)

save(user)

Database
Database

user

user_id	email	first_name	last_name	role
1	hacker@evil.com	Attacker	User	ADMIN

user

user_id	email	first_name	last_name	role
1	hacker@evil.com	Attacker	user	ADMIN

DB State Change: Role Escalated

Privilege Escalation
Complete



Escalated to Admin

위협 사례 2

CORS 보안 정책 무력화

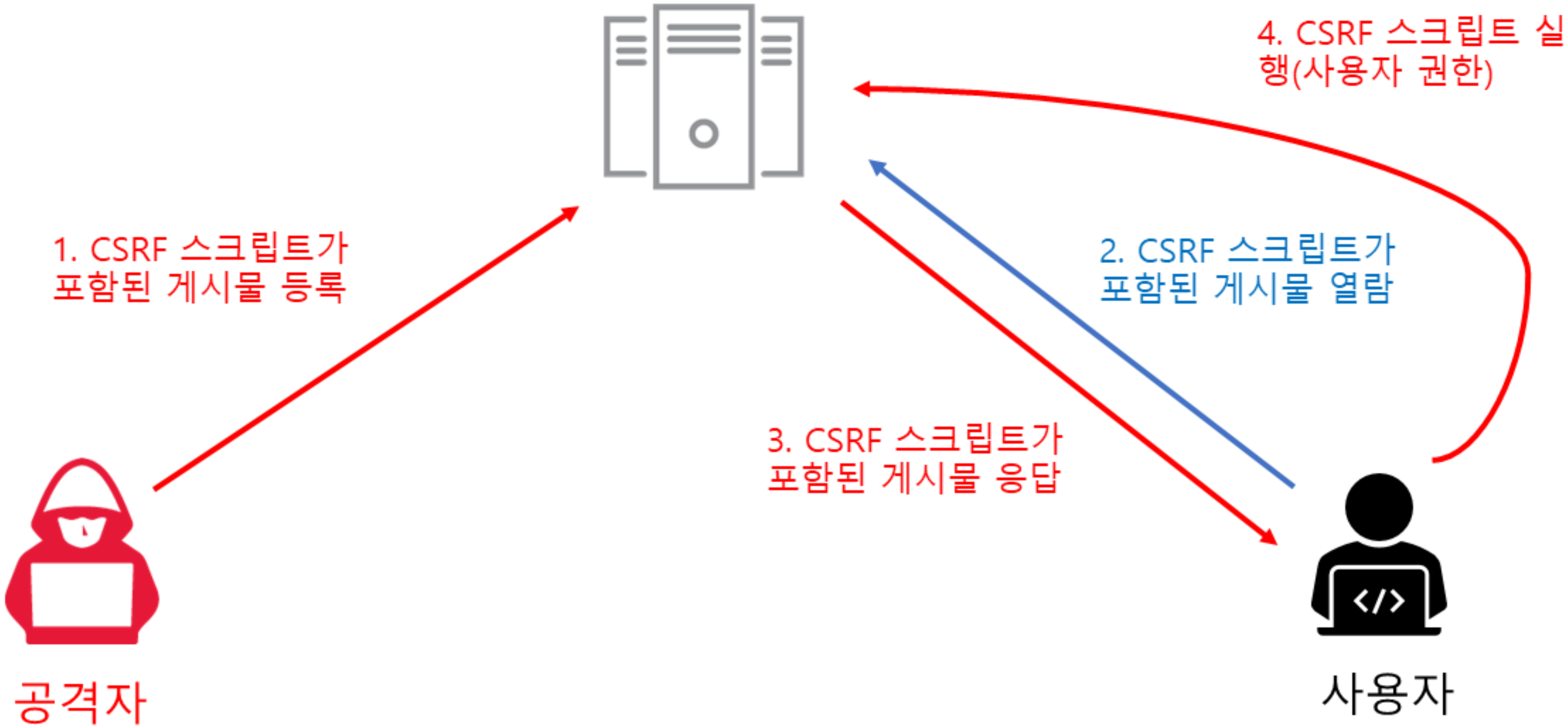
프론트엔드와 API 연동 시 발생하는 CORS 에러 해결을 요구하면, AI는 브라우저 출처를 허용하는 코드(`allow_origins=[*]`)를 제시함

운영 환경에 배포될 경우 브라우저 보안 정책이 무력화되며, 사용자가 악성 클릭 시 세션 쿠키를 통째로 탈취당하는 CSRF 공격이 이루어질 수 있음

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],           # 1. 모든 도메인 접속 허용
    allow_credentials=True,       # 2. 인증 정보(세션 쿠키)까지 포함하도록 허용
    allow_methods=["*"],
    allow_headers=["*"],
)
```



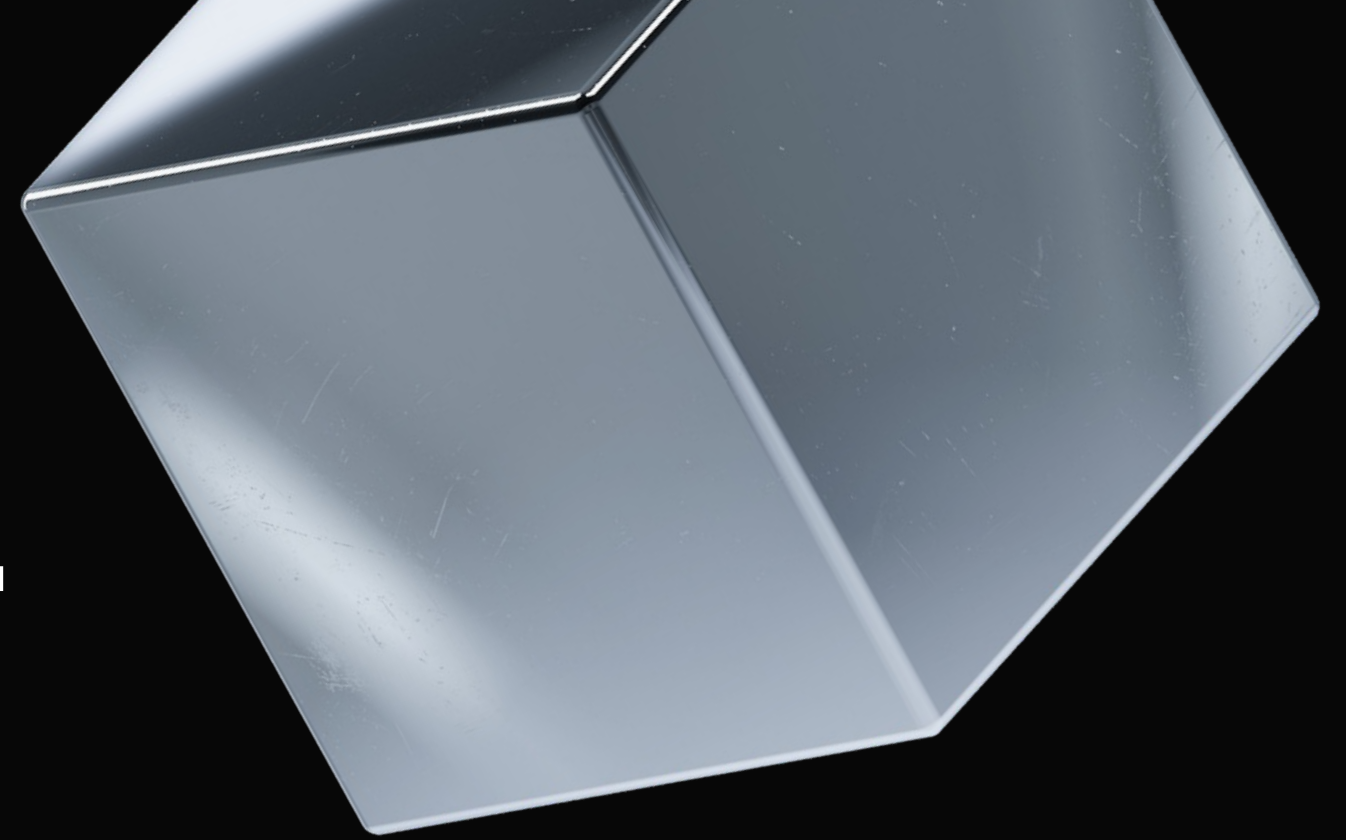
운영 서버에서 살아남기 : DevSecOps 4중 자동화 방어망



개인 주의력의 한계와 Shift-Left Security

AI가 10초 만에 쏟아내는 수백 줄의 코드를 인간의 코드 리뷰만으로 100% 검증하는 것은 불가능함

보안 검증 단계를 배포 직전이 아닌 '코드 개발 및 병합' 초기 단계로 앞당기는 시스템적 강제가 필요함



CI/CD 기반 4중 필터 구축

Semgrep

AI가 작성한 코드를 실행
없이 정적 분석하여 SQL
인젝션, XSS, 논리적
결함을 컴파일 전에 차단

Dependency-Check

AI가 기능 구현을 위해
무심코 추가한 외부
오픈소스 라이브러리의
알려진 취약점(CVE) 스캔

Trivy

운영 환경과 동일하게
구성된 도커 이미지 내부의
OS 레벨 취약점 및
Dockerfile 권한 설정 오류
탐지

Gitleaks

프롬프트에 코드를
복사/붙여넣기 하다가
실수로 하드코딩된
API키나 비밀번호가
깃허브에 커밋되는 휴먼
에러 차단

Files

<

🔗 dev

+

🔍

🔍

Go to file

t

▼

📁 .github/workflows

📄 cd.yml

📄 ci.yml

📄 security-spring.yml

> 📁 gradle

> 📁 src

> 📁 uploads

📄 .dockerignore

📄 .gitattributes

📄 .gitignore

📄 build.gradle

📄 docker-compose.yml

📄 dockerfile

📄 gradlew

📄 gradlew.bat

📄 settings.gradle

MJSEC_LMS_BACK / .github / workflows / security-spring.yml

↑ Top

Code

Blame

307 lines (272 loc) · 12.6 KB

📄

👤

Raw

📄

📄

✎

▼

🔗

```
46      # -----
47      # Semgrep (SAST)
48      # -----
49      - name: Set up Python (for Semgrep CLI)
50        uses: actions/setup-python@v5
51        with:
52          python-version: '3.11'
53
54      - name: Install Semgrep
55        run: |
56          python -m pip install --upgrade pip
57          pip install semgrep
58
59      - name: Run Semgrep (OWASP + Java/Kotlin) → SARIF
60        run: |
61          set -euo pipefail
62          semgrep --version
63          semgrep ci \
64            --config p/owasp-top-ten \
65            --config p/java \
66            --config p/kotlin \
67            --sarif -o semgrep.sarif || true
68
69      - name: Run Semgrep (JS/React) → append SARIF
70        if: ${ hashFiles('**/package-lock.json', '**/npm-shrinkwrap.json', '**/pnpm-lock.yaml', '**/yarn.lock') != '' }
71        run: |
72          set -euo pipefail
73          semgrep ci \
74            --config p/javascript \
75            --config p/react \
76            --sarif -o semgrep-js.sarif || true
77          jq -s '.[0].runs += .[1].runs | .[0]' semgrep.sarif semgrep-js.sarif > semgrep-merged.sarif && mv semgrep-merged.sarif semgrep.sarif
78
79      - name: Check Semgrep SARIF has results
80        id: semgrep_sarif_check
81      -
```



진화하는 시대의 코어 역량

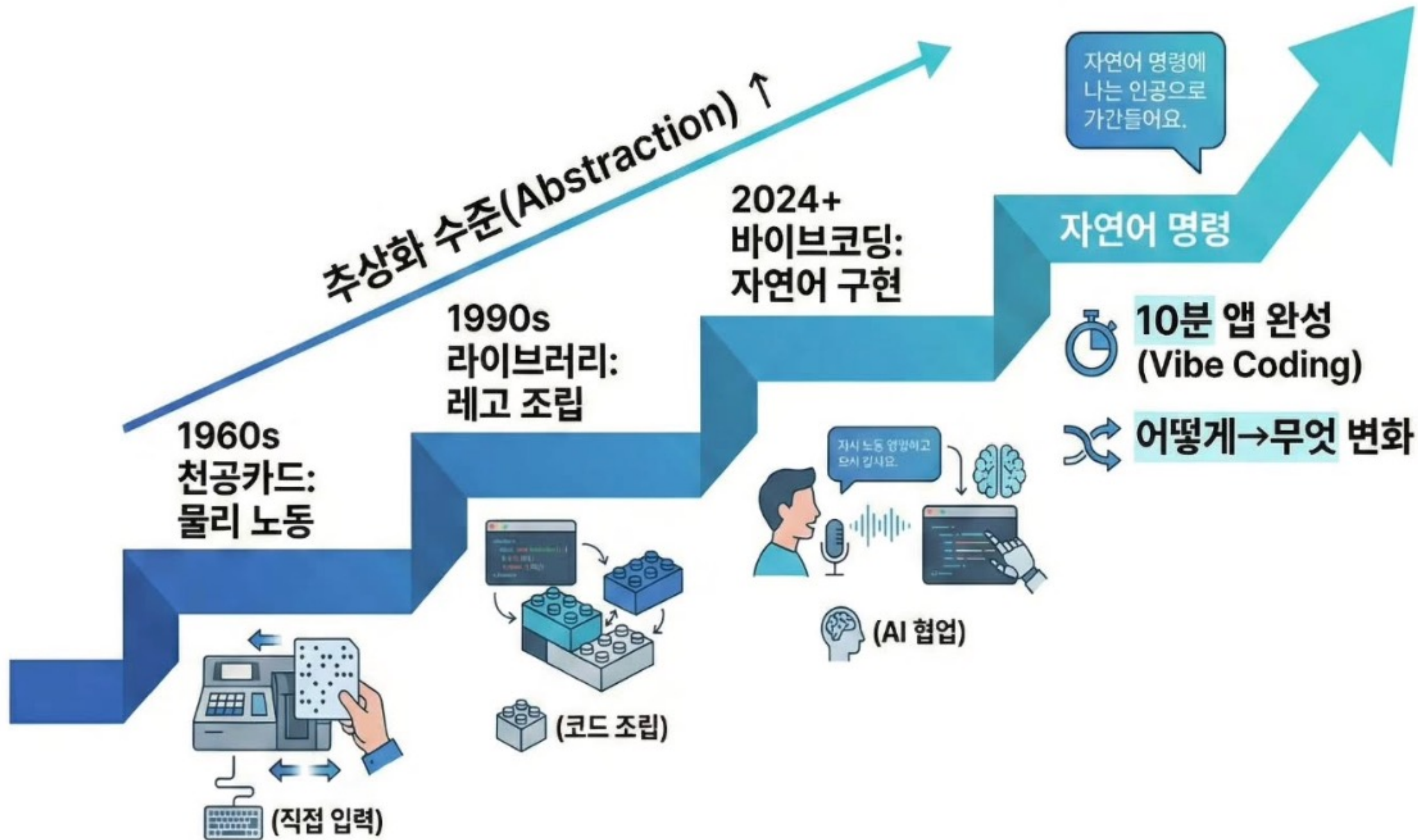


기술의 발전은 편의성에 초점이 맞추어져 있음

(1) 기계어 -> 어셈블리 -> C/C++

(2) C언어에서 메모리를 할당하던 시절을 지나
Java의 Garbage Collector로 메모리 관리를 추상화





단순히 남들보다 코드를 빠르게 생산해 내는 것은
더 이상 독보적인 무기가 될 수 없음

AI가 쏟아내는 무수한 코드를 DevSecOps라는
파이프라인을 통해 통제해야 함



시스템 전체의 동작 원리를 가장 낮은 레벨부터 깊이 이해

어떤 예외 상황과 트래픽 앞에서도 시스템이 무너지지 않도록
안전한 아키텍처를 설계

코드에 대한 책임감 + 개발하는 도메인에 대한 이해





질문이 있으신가요?

이메일 : mspark010506@gmail.com